

Teaching a Robot to Solve Mazes

A plain-language case study in reinforcement learning — the story of "Leaper," runs PPO_1 to PPO_26

This is not a chronological diary. It's a set of **lessons**. Over 26 experiments we tried to teach a six-legged robot to walk across a cluttered field and reach a goal without getting stuck. Most experiments *failed* — and the failures are the valuable part, because each one taught us a rule about how learning machines actually behave. Read this whenever you're building something that learns and you want to remember what really matters.

FIRST, THE VOCABULARY (THAT'S ALL YOU NEED)

- **Agent** — the robot's "brain." It looks at the situation and picks an action.
- **Reward** — a score we hand out. The agent's *only* goal in life is to collect more of it. It does not know what we *meant* — only what we *rewarded*.
- **Episode** — one attempt: start somewhere, try to reach the goal, until you arrive or run out of time.
- **PPO** — the training method. It nudges the brain, over millions of tries, toward actions that earned more reward. Think "trial and error, at scale, with good bookkeeping."
- **The exam** — 100 fixed mazes the robot never trained on. Its score = how many it solves. Same exam every run, so we can compare fairly.

HOW THE LEARNING ACTUALLY HAPPENS (IN ONE PARAGRAPH)

The robot plays the same kind of game millions of times. Each attempt (an *episode*) produces a trail of "situation → action → reward" moments. PPO looks back over big batches of these and asks a simple question of every action: *did things tend to go better or worse after I did that?* Actions that were followed by more reward get made slightly more likely next time; actions followed by less reward get made slightly less likely. Repeat that nudge a few million times and a competent behavior slowly crystallizes out of pure trial and error. Nobody ever tells it *how* to solve a maze — we only shape *what earns reward*, and it discovers the "how" itself. This is why the reward is so powerful, and so dangerous: it is the only instruction the robot ever truly receives.

The target throughout: **solve 85% of the exam mazes**. Here's how we got there — and everything we learned failing along the way.

LESSON 1

You cannot act on what you cannot see

Our first robots were **blind**. They knew the direction and distance to the goal (like a compass), but nothing about the obstacles around them. We rewarded getting closer to the goal and penalized crashing.

WHY IT FAILED — PPO_8 TO PPO_10

A blind robot near an obstacle has no good move. Pushing toward the goal just means crashing over and over. So it learned the only "safe" thing available: **stop moving entirely** and wait out the clock. PPO_8 stood still on 98.85% of steps and scored **0%**. Freezing wasn't a bug in the code — it was the *smartest* thing a blind robot could do to avoid crash penalties. The problem wasn't the brain. It was the lack of eyes.

WHAT "SEEING" ACTUALLY MEANS TO THE ROBOT

The robot never gets a picture. Its entire view of the world is a short list of numbers, updated every step. In plain terms it knows:

- **A compass to the goal** — the direction and distance to the target (it always knows *where* the goal is; the hard part is getting *there*).
- **Its own facing and position** in the arena.
- **What it just did and whether it just crashed** — a little short-term self-awareness.
- **The rays** — the crucial addition: several "tape-measure" beams fanning out ahead, each reporting how far it is to the nearest obstacle in that direction.

We gave it those eyes: eight rays that report the distance to the nearest obstacle in several directions (**PPO_11**). We never told it "avoid obstacles" — we just let it *see* them and kept the same crash penalty. It figured out avoidance on its own, and the score leapt from 5% to **69%**. Notably, this was the *only* change: same brain, same reward, same everything — just eyes added. That's what makes it clean evidence that **perception**, specifically, was the missing piece.

KEY IDEA

Perception was the single biggest win in the entire project. Before you make a learning agent "smarter," ask whether it can even *perceive* the thing it's failing at. Most "stupid" behavior is really **blindness** — a rational response to missing information.

LESSON 2

The reward is a wish — and wishes backfire

Even with eyes, the robot sometimes wedged against a rock and re-tried the same blocked move forever, timing out. The obvious fix seems to be: "punish crashing harder." So we tripled the crash penalty (**PPO_15**).

THE WHOLE REWARD, SPELLED OUT

The robot's entire sense of "good" and "bad" is just four ingredients added together each step. This tiny recipe is the only instruction it ever gets:

- **+ Progress** — a little reward for every unit it gets *closer* to the goal (and a matching loss for moving away). This is the engine that pulls it forward.
- **– Time** — a tiny cost every step, so dawdling is mildly discouraged.
- **– Crash** — a penalty for any step where it collides.
- **+ Goal** — a big one-time bonus for actually reaching the target.

That's it. Every clever or foolish thing the robot ever does is it chasing *this* sum. Notice there is no term that says "explore," "be brave," or "don't get stuck" — which is exactly why those behaviors had to be *engineered into the reward* later.

WHY IT FAILED — PPO_15

A bigger crash penalty didn't teach *better navigation* — it taught **fear**. The robot became timid: it moved forward far less (74% → 46% of steps) and still got wedged. We asked for "crash less" and it heard "move less." Success didn't budge. We reverted it.

The real fix (**PPO_16**) came from a sharper idea. The robot kept getting stuck because *nothing bad happened fast enough* when it wedged — it just slowly leaked tiny time penalties. So we added a **stuck rule**: if you collide with no progress for 40 steps in a row, the episode *ends immediately* with a one-time penalty. Crashes fell from 15.87% to **1.35%** and the freezing vanished.

KEY IDEA — THE SUBTLE PART

Ending an episode early is *not automatically a punishment* — if you just "end early to escape a bad spot," you might be **rewarding** the robot for quitting. That's why the stuck rule needed an explicit penalty *and* had to end the episode as a true failure. The lesson: **the agent optimizes exactly what you reward, not what you intended**. When behavior is weird, don't blame the robot — re-read your reward as if a clever lawyer wrote it.

LESSON 3

How you move must match how you see

We wanted human-like vision: instead of seeing in all directions (360°), see mostly *forward* — a 270° cone, blind only directly behind. We first tried this while still letting the robot **reverse** (PPO_17). It scored a miserable **8%**.

WHY IT FAILED — PPO_17

We gave it narrow forward vision but let it back up into the exact region it *couldn't see*. It kept reversing blindly into obstacles and, unsure of itself, dithered. The two design choices **contradicted each other**.

The fix (PPO_18) was to make the design *coherent*: see forward **and** only walk forward (turn to face where you want to go, then drive). Now the robot always travels into what it can see — exactly how a person walks. Score jumped to **75%**.

KEY IDEA

A capable agent needs its parts to **agree**. Vision and movement, sensors and actions, reward and goal — a great sensor paired with a contradictory ability is worse than either alone. Design the whole loop, not one piece at a time.

INTERLUDE

A result is only as honest as the test is hard

Midway through, we moved the robot from a small arena into a big, open field with scattered obstacles (PPO_13). It scored a gorgeous **85%** with almost zero crashes. We could have declared victory right there. We didn't — and that instinct mattered.

WHY THE 85% WAS MISLEADING — PPO_13

The field was *roomy*. With the same-size robot in a much larger space, there was simply a lot of empty room to wander through without hitting anything. The near-perfect crash rate was partly **the geometry of an easy room**, not proof of skill. A flattering score on an easy test tells you very little.

So we deliberately made the test *harder* (PPO_14): we packed the field with **51 obstacles**, shrank the open space, and moved the goal into an awkward corner. The score honestly collapsed to **54%** and the old freezing problem came roaring back. That "worse" number was far more *useful* than the 85% — it exposed the real weakness that the easy field had been hiding. This dense arena became the fair, punishing exam every later run was measured on.

KEY IDEA

Design your test to *attack* your system, not flatter it. A hard, honest benchmark that makes your score drop is a gift: it shows you the real problem while you can still fix it. Chasing a pretty number on an easy test is how projects fool themselves for months.

LESSON 4

A bigger brain is not more skill

Stuck at 75%, the robot's remaining failures were **timeouts**: it would wander into a dead-end pocket that pointed toward the goal, then never find its way out. That *looks* like a thinking problem. So we tried to make it think harder — twice.

WHY THEY FAILED — PPO_19-21 (MEMORY) AND PPO_22 (BIGGER BRAIN)

- **Memory (LSTM):** we gave it a brain that remembers where it has been, so it could recall a dead-end. Result: 71%, 69% — *below* champion, at every setting we tried.
- **Bigger brain:** we made the network 16× larger (256×256 vs 64×64). Result: 74% — a statistical tie. No change.

KEY IDEA — HOW TO FIND THE REAL BOTTLENECK

Two separate "smarter brain" upgrades did **nothing**. That absence of improvement *was the clue*: if extra thinking-power doesn't help, then thinking wasn't the limit. The bottleneck had to be somewhere else — what the robot could **see**. In machine learning, a change that *doesn't* move the needle is data, not a dead end. It tells you where *not* to look next.

LESSON 5

Resolution beats reach — the vision lesson

So we returned to the eyes. There are two different ways to "improve vision," and they are **not** the same:

Winner: more rays (PPO_23) — sharper vision

We doubled the number of rays from 8 to 16 across the same forward cone. Eight rays left big blind gaps *between* the beams, where a whole obstacle or a fake "doorway" could hide. Sixteen rays gave a clearer read of where real openings were. Score rose to a best of **87%** — timeouts cut nearly in half. More rays = **genuinely new information**.

Loser: longer range (PPO_24 & PPO_26) — and here's the surprising part

Intuitively, seeing *farther* (range 28 → 45) should help. It made things **worse** — every time. To understand why, you need to know how a ray reports distance.

THE MECHANISM — WHY LONGER SIGHT HURT

A ray doesn't report "wall at 14 units." It reports a **fraction**: $\text{distance} \div \text{max-range}$. The max-range is the divisor. So take the *same* obstacle 14 units ahead:

- At range 28: reading = $14 \div 28 = \mathbf{0.50}$
- At range 45: reading = $14 \div 45 = \mathbf{0.31}$

Same obstacle — but at range 45 the signal is **fainter**. Each ray has only "one number's worth" of precision. Stretch the range and you spread that precision over a bigger distance, so the **close-up world — where decisions are actually made — gets blurrier**. Worse, in a dense field almost every long ray now hits *some* distant clutter, so open doorways stop reading as clearly open. The robot drowns in far-away obstacles it would have turned away from anyway, and hesitates.

KEY IDEA — THE DISTINCTION WORTH MEMORIZING

More rays = more real data (new directions, new information). **More range = the same data stretched thinner** (a trade, not a gift). "Better sensing" is not one dial. Adding *information* helps; stretching a *fixed* signal to cover more ground trades away the precision you needed most.

Like headlights: more lamps let you see in more directions (good). Swapping your normal beam for a dim, extra-long spotlight lets you see farther but makes the thing three feet ahead harder to judge — and lights up distant scenery you'll never reach (bad trade).

LESSON 6

One run is not proof — the seed lottery

PPO_23's "87%" hid a trap. We re-ran the *exact same recipe* with different random starting conditions ("seeds") and got: **86, 73, 87, 80**. Same recipe. Four very different robots.

WHY THE SAME RECIPE GIVES DIFFERENT RESULTS

Training starts from randomness — which maze comes first, how the brain is initially wired. Those dice-rolls push the robot down different learning paths. Here the split was about **nerve**: some seeds trained *bold* (kept moving, ~87%, cleared target); others trained *timid* (dithered and stopped, ~73–80%). The recipe wasn't a single score — it was a **range**.

KEY IDEA

Never trust one training run. A single result is one sample from a distribution. Always re-run with several seeds and report the **spread**, not just the best. A "win" that only happens on a lucky seed isn't a win — it's a coin that sometimes lands well.

LESSON 7

Fix the behavior, not just the brain

The variance *was* the problem now — not intelligence, not perception, but that the robot randomly came out bold or timid. The insight: **standing still was almost free**. A timid robot near a tricky spot paid nearly nothing to just... loiter until the clock ran out. So loitering was a safe habit to fall into.

The fix (**PPO_25**): a small **idle penalty** — a tax on standing still. But the *size* of that tax is the whole art:

SIZING AN INCENTIVE — THE GOLDBLOCKS REASONING

- **Too small** ($\approx$ the 0.01 time cost): loitering stays free. No effect.
- **Too big** ($\approx$ the 0.18 crash penalty): now standing hurts as much as crashing, so the robot *rams walls* just to avoid standing. You'd trade timeouts for collisions.
- **Just right (0.04)**: about 4 $\times$ a normal step but only $\frac{1}{4}$ of a crash. Loitering clearly costs something; moving is still worth it. We also gave a **3-step grace** so a brief pause to pivot ("thinking") stays free — only *sustained* standing is taxed.

Put in a table, the reasoning is just "find the number that lives between the two costs already in the reward":

Penalty for standing still	Compared to...	What the robot learns
0.01 (= the time cost)	same as a normal step	Loitering stays free → no change
0.04 (chosen)	4× a step, ¼ of a crash	Move, but don't be reckless → fixed
0.18 (= the crash cost)	as bad as hitting a wall	Rams walls to avoid standing → worse

The result was the cleanest win of the project — across four seeds:

	Before (PPO_23)	After idle penalty (PPO_25)
Seed scores	86, 73, 87, 80	89, 92, 93, 92
Average	81.5%	91.5%
Spread (the lottery)	14 points (73–87)	4 points (89–93)
Seeds clearing 85%	2 of 4	4 of 4

KEY IDEA

The last 15 points of skill came not from a smarter or better-seeing robot, but from **making a bad habit cost something**. Two things happened at once: the average jumped +10 points, *and* the lottery died — every run now comes out reliably good. When an agent behaves inconsistently, look for a "free" bad option in the reward and put a price on it.

LESSON 8

Trust, but verify — did it really learn?

A 93% on 100 mazes is encouraging, but 100 is a small sample — the "true" skill could be anywhere from about 88% to 98%. And maybe those 100 mazes were just easy. So we ran two extra checks:

- **A bigger, fresh exam:** we graded the champion on **1,000 brand-new mazes** from a completely different set it had never seen. It scored **91.3%** — essentially the same. If it had merely *memorized* the original 100, this fresh test would have collapsed. It didn't.
- **A tighter confidence range:** 1,000 mazes shrink the uncertainty from "±5%" to "±1.7%." So the honest, defensible number is **~91%**, floor still above target — not a fragile 93%.

KEY IDEA

A score on a small test is a rumor; a score on a big *fresh* test is evidence. Always test on data the model never trained on (that's the difference between **learning** and **memorizing**), and always ask how *wide* your uncertainty is before you trust the last digit.

The whole journey at a glance

Run	What we changed	Score	Lesson
PPO_1–2	Baseline, blind, body-only	~60%*	Starting point (easy arena)
PPO_8–9	Harder reward, still blind	0%	A blind agent freezes rationally
PPO_10	Allow reverse/stop	5%	Fixed the literal freeze, not skill
PPO_11	Added 8 vision rays	69%	Perception = biggest win (L1)
PPO_12	Can turn/reverse out of contact	65%	More escape routes
PPO_13	Bigger random field	85%	...but field was open (misleading)
PPO_14	Dense 51-obstacle field	54%	The real, hard problem appears
PPO_15	Tripled crash penalty	51%	Bought fear, not skill (L2)
PPO_16	Stuck rule (end + penalty)	64%	Reward design fixed the freeze (L2)
PPO_17	Forward vision, but still reverse	8%	Vision & movement contradicted (L3)
PPO_18	Human vision + forward-only	75%	Coherent design (L3)
PPO_19–21	Memory (LSTM)	69–71%	More brain ≠ more skill (L4)
PPO_22	16× bigger network	74%	Capacity wasn't the bottleneck (L4)
PPO_23	8 → 16 rays (sharper)	81.5% avg	Resolution helps (L5) — but a lottery (L6)
PPO_24	Range 28 → 45 (longer)	80%	Stretched signal too thin (L5)
PPO_25	Idle penalty (anti-dither)	91.5% avg	Fix behavior; kill variance (L7)
PPO_26	Longer range + idle penalty	82%	Confirmed longer range hurts (L5)

*PPO_1–2 ran on a much easier early arena and are not directly comparable to the later 100-maze exam; shown for context only.

The meta-lessons (the ones that transfer to anything)

CARRY THESE TO YOUR NEXT PROJECT

1. **Perception first.** "Stupid" behavior is usually blindness. Check what the agent can sense before you make it smarter.
2. **The reward is law.** The agent does what you *reward*, not what you *mean*. Weird behavior = re-read your reward literally.
3. **Design coherently.** Parts must agree. A great sensor with a contradictory ability is worse than neither.
4. **A failed upgrade is information.** When more brain/memory does nothing, that *rules out* a whole direction and points elsewhere.
5. **Know your dials.** "More sensing" isn't one thing — adding information $\neq$ stretching a fixed signal thinner.
6. **Never trust one run.** Report the spread across seeds, not the luckiest number.
7. **Put a price on free bad habits.** Inconsistency often hides a zero-cost lazy option; taxing it can beat any architecture change.
8. **Verify on fresh, large tests.** Learning vs. memorizing is only visible on data the model never saw.
9. **Make the test hard on purpose.** A flattering score on an easy benchmark hides the real problem. A hard test that lowers your number is doing you a favor.
10. **Change one thing at a time.** If you move several dials at once, a change in results can't be traced to a cause. Clean evidence comes from clean experiments.

A NOTE ON HOW WE RAN THE EXPERIMENTS

The lessons above only became visible because of the **discipline** in how runs were done. Three habits did most of the work:

- **Change one thing at a time.** Almost every run altered a single variable and kept everything else identical. When several things change at once and the score moves, you can't tell *which* change did it — the evidence is muddy. (The few "bundle" runs are flagged honestly as inconclusive.)
- **Keep the exam fixed.** The same 100 held-out mazes graded every run, so any change in score came from the *robot*, never the test.
- **A short smoke test before the long run.** A tiny throwaway run first, just to confirm the pipeline works, before committing to a multi-hour training run. Cheap insurance against wasting an afternoon on a typo.

Final champion: **PPO_25** — 16 rays, 270° forward cone, range 28, forward-only movement, idle-penalty 0.04 with a 3-step grace. **~91% on 1,000 unseen mazes.** Built not by making the robot

think harder, but by helping it *see* the right amount and refusing to let it *stand still*.

Leaper reinforcement-learning case study · runs PPO_1–PPO_26 · compiled 2026-08-27. Companion technical log: RUN_LOG_PPO23-26.md.